K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Basics of Neural Network & Deep Learning
ASSIGMENT

SUBMITTED BY:

Diwakar Gupta
2301201014
BCA (AI&DS)
SECTION -A

SUBMITTED TO:

Mr. Anoja Mishra

1. What are epoch, batch size, and Stratified? Please explain them.

ANSWER.

Epoch:
An epoch in machine learning is one full run through the training dataset, during which each data sample is run through the model and its parameters are adjusted in response to the computed error. Multiple epochs are needed for the training process, which enables the model to improve iteratively by changing its parameters in response to the computed error.

Batch Size:
In neural networks, the training data is often divided into smaller batches, each of which is processed independently before the model parameters are updated. The batch size refers to the number of samples used in each of these smaller batches during training.
During training, the model makes predictions for all the data points in the batch and compares them to the correct answers. The error is then used to adjust the model's parameters using Gradient Descent. The batch size is one of the key hyperparameters that can influence the training process, and it must be tuned for optimal model performance.
There are three primary types of gradient descent associated with batch size:

1. Batch Gradient Descent (BGD): Uses the entire dataset in one iteration. The batch size is equal to the total number of training samples.
2. Mini-Batch Gradient Descent (MBGD): Uses a predefined number of samples from the dataset for each update. This method lies between batch and stochastic gradient descent (SGD).
3. Stochastic Gradient Descent (SGD): Processes only one sample at a time for each update, making the batch size equal to 1.

Stratified:
Splitting the dataset in such a way that each class is represented in the same proportion in both training and testing data.
In classification problems, your data may be imbalanced (one class has more samples than another).
If you split randomly:

- One class might dominate the training set
- Another class might be missing in the test set

2. Explain the different types for activation functions?

ANSWER.

Activation functions decide whether a neuron should be activated or not.
They introduce non-linearity, which helps the neural network learn complex patterns.
Main Types of Activation Functions
1. Sigmoid Function
- Output range: 0 to 1
- Formula:

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Uses:
- Binary classification (output layer)

Problems:
- Vanishing gradient
- Slow training

2. Tanh (Hyperbolic Tangent)
- Output range: -1 to 1

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Advantages:
- Zero-centered
- Better than sigmoid

Problem:
- Still suffers from vanishing gradient

3. ReLU (Rectified Linear Unit)
- Output:
  - 0 if x < 0
  - x if x ≥ 0

$$f(x) = \max(0, x)$$

Advantages:
- Fast and efficient
- Most commonly used

Problem:
- "Dead neurons" (neurons stop learning)

4. Leaky ReLU
- Fixes ReLU problem
- Small slope for negative values

$$f(x) = \begin{cases} x & x > 0 \\ 0.01x & x \leq 0 \end{cases}$$

Advantage:

- Prevents dead neurons

5. Softmax Function

- Converts output into probabilities
- Output values sum to 1

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

Uses:

- Multi-class classification (output layer)

3. How do we handle structured and unstructured data in ANNs and CNNs?

ANSWER.

Unstructured Data:
When dealing with unstructured data, neural networks must first transform it into a structured format that the network can process. For instance, CNNs interpret images by breaking them down into pixels and analyzing the patterns and features within these pixels. This allows the network to understand and learn from visual information. Similarly, RNNs handle sequences, like sentences in text, by considering the order of elements and their context, which is crucial for understanding language. The preprocessing steps for unstructured data are thus more complex, often involving techniques like tokenization for text or feature extraction for images.

Structured Data:
In contrast, structured data is more straightforward for neural networks to interpret. Datasets with clear numerical or categorical values fit easily into the framework of a neural network without the need for extensive preprocessing. The network can directly learn from the structured inputs, with each node in the input layer representing a specific variable from the dataset. This direct mapping simplifies the learning process, as the network can focus on finding patterns and relationships between the variables without the additional step of data transformation.

Preprocessing Needs:
Preprocessing is a critical step in preparing both structured and unstructured data for neural networks. For structured data, preprocessing may involve normalization or one-hot encoding to make the data more suitable for the network. Unstructured data preprocessing is more involved; it includes steps like converting text to

numerical vectors using techniques such as word embeddings or transforming audio files into spectrograms. These preprocessing steps are crucial for enabling neural networks to identify patterns and make predictions based on unstructured inputs.

Network Architecture
The architecture of a neural network is heavily influenced by the type of data it processes. For unstructured data, architectures like CNNs and RNNs are tailored to handle the complexity and variability of the data. CNNs use convolutional layers to filter inputs and capture spatial hierarchies, making them ideal for image recognition. RNNs, with their feedback loops, are suited for sequential data like text or time series. Structured data typically uses simpler architectures, as the data's format does not require such specialized processing layers.

Learning Differences:
The learning process within neural networks also differs between structured and unstructured data. With structured data, neural networks can employ straightforward backpropagation to adjust weights and minimize error. Unstructured data often requires more sophisticated learning techniques. For example, CNNs use pooling layers to reduce dimensionality and improve feature detection, while RNNs utilize gates to manage information flow over sequences. These mechanisms help neural networks to learn effectively from unstructured data's complexity.

4. What is a layer? Define ANN and CNN layers?

ANSWER.

A layer is a group of neurons (nodes) that process input data and pass the output to the next layer.A layer is a step in the neural network where data is processed.

An ANN consists of three primary types of layers:

- Input Layer
- Hidden Layers
- Output Layer

1. Input Layer
Input layer is the first layer in an ANN and is responsible for receiving the raw input data. This layer's neurons correspond to the features in the input data. For example, in image processing, each neuron might represent a pixel value. The input layer doesn't perform any computations but passes the data to the next layer.

2. Hidden Layers
Hidden Layers are the intermediate layers between the input and output layers. They perform most of the computations required by the network. Hidden layers can vary in number and size, depending on the complexity of the task.
Each hidden layer applies a set of weights and biases to the input data, followed by an activation function to introduce non-linearity.

3. Output Layer
Output Layer is the final layer in an ANN. It produces the output predictions. The number of neurons in this layer corresponds to the number of classes in a classification problem or the number of outputs in a regression problem.

5. What concepts are used in ANN and CNN?

ANSWER.

Both ANN (Artificial Neural Network) and CNN (Convolutional Neural Network) share some common concepts, but CNN also has some special ones.

Common Concepts (ANN + CNN)
1. Neurons (Nodes)
- Basic units of a network
- Receive input, process it, and pass output

2. Weights and Bias
- Weights: importance of each input
- Bias: helps shift the output

3. Activation Functions
- Add non-linearity
- Examples: ReLU, Sigmoid, Softmax

4. Forward Propagation
- Data moves from input → hidden → output

5. Loss Function
- Measures error between actual and predicted output

6. Backpropagation
- Updates weights to reduce error

7. Gradient Descent
- Optimization technique to minimize loss